Hızlı ve öğrenmesi kolay olduğu için go dilisni tercşh ettik,
Go dilindeki fiber framework'ünü tercih ettik, nodejs'deki express'e benzer bir yazım şekli var.
Daha yönetilebilir ve güncel olduğu için postgresql'i tercşh ettik.

```go
func main() {
    cfg := config.Instance()
    dbConnection()

    app := getApp()

    // cors Settings
    app.Use(cors.New())

    // /
    app.Get("/ping", func(c *fiber.Ctx) error {
        return c.SendString("pong")
    })

    // /API/
    apiRouter := app.Group("/api")
    apiRouter.Get("/ping", func(c *fiber.Ctx) error {
        return c.SendString("api pong")
    })

    // /API/AUTH/
    authRouter := apiRouter.Group("/auth")
    authRouter.Post("/login", auth.Login)
    authRouter.Post("/signup", auth.Signup)
    authRouter.Post("/refresh", auth.Refresh)
    authRouter.Get("/verify/:token", auth.Verify)

    log.Fatal(app.Listen(":" + strconv.Itoa(cfg.Port)))
}
```

Body verilerini kontrol için validator kütüphanesi kullandık (
github.com/go-playground/validator/v10)

```go
)

type (
    SignupPayload struct {
        Name        string `validate:"required,min=3,max=50"` //
        Surname     string `validate:"required,min=3,max=50"`
        Username    string `validate:"required,min=6,max=16"`
        Email       string `validate:"required,email,max=255"`
        Password    string `validate:"required,min=8,max=255"`
        PhoneNumber string `validate:"required,numeric,len=10"`
    }
```

```go
func (v *XValidator) Validate(data interface{}) string {
    validationErrors := []ErrorResponse{}

    errs := v.validator.Struct(data)
    if errs != nil {
        for _, err := range errs.(validator.ValidationErrors) {
            var elem ErrorResponse
            elem.FailedField = err.Field()
            elem.Tag = err.Tag()
            elem.Value = err.Value()
            elem.Error = true
            validationErrors = append(validationErrors, elem)
        }
    }

    errMsgs := make([]string, 0)

    for _, err := range validationErrors {
        errMsgs = append(errMsgs, fmt.Sprintf(
            "[%s]: '%v' | Needs to implement '%s'",
            err.FailedField,
            err.Value,
            err.Tag,
        ))
    }

    return strings.Join(errMsgs, "\n")
}
```

Mail için go’nun kendi kütüphaneleri olan net/smtp ve html/template (html templateleri ile daha okunabilir kod yazmak için) kütüphaneli kullandık. (smtp servisini ücretsiz bir şekilde googledan aldık (bahsedilmesi opsiyonel)

```go
func Send(template templates.MailTemplate) error {

    cfg := config.Instance()

    from := cfg.SMTPUser
    password := cfg.SMTPPassword
    host := cfg.SMTPHost
    port := cfg.SMTPPort

    auth := smtp.PlainAuth("", from, password, host)

    body, errTemplate := template.Content()
    if errTemplate != nil {
        return errTemplate
    }
    errSendMail := smtp.SendMail((host + ":" + strconv.Itoa(port)), auth, from, []string{template.To()}, body.Bytes())
    if errSendMail != nil {
        log.Println("SEND MAIL ERROR", errSendMail)
        return errSendMail
    }
    fmt.Println("Successfully sent mail")
    return nil
}
```

Veritabanını tasarlarken veritabanının kullanılabilir özelliklerini kullandık (enumlar, triggerlar, fonksiyonlar)

```sql
CREATE TABLE users.profile (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    username varchar(50) NOT NULL UNIQUE,
    phone_number varchar(20) NOT NULL UNIQUE,
    email varchar(255) NOT NULL UNIQUE,
    password_hashed varchar NOT NULL,
    "name" varchar(100) NOT NULL,
    surname varchar(100) NOT NULL,
    contact_email varchar(255) NULL,
    contact_phone varchar(20) NULL,
    profile_url varchar NOT NULL DEFAULT 'default.png',
    created_at timestamptz NOT NULL DEFAULT NOW(),
    updated_at timestamptz NOT NULL DEFAULT NOW(),
    is_verified boolean NOT NULL DEFAULT false,
    verification_token varchar NULL,
    verification_token_exp timestamptz NULL,
    resetp_token varchar NULL,
    resetp_exp timestamptz NULL,
    is_2fa_enabled boolean NOT NULL DEFAULT false,
    user_status users.user_status_enum NOT NULL DEFAULT 'pending_verification'
);
```

Kullanıcı doğrulaması için jwt token sistemini kullandık.

```go
func Create(username string, which bool) (string, error) {
    cfg := config.Instance()
    var jwtValues TokenClaim
    var key string
    if which {
        key = cfg.JWTRefreshKey
        jwtValues = crateClaim(username, false)
    } else {
        key = cfg.JWTAccessKey
        jwtValues = crateClaim(username, true)
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwtValues)
    tokenString, err := token.SignedString(([]byte)(key))
    if err != nil {
        log.Println(err)
        return "", errors.New("token cannot be created")
    }
    return tokenString, nil
}
```

```go
func VerifyToken(tokenString string, which bool) (TokenClaim, error) {
    cfg := config.Instance()
    var jwtKey string
    if which {
        jwtKey = cfg.JWTAccessKey
    } else {
        jwtKey = cfg.JWTRefreshKey
    }
    token, err := jwt.ParseWithClaims(tokenString, &TokenClaim{}, func(token *jwt.Token) (any, error) {
        return []byte(jwtKey), nil
    })
    if err != nil {
        log.Println(err)
        return TokenClaim{}, err
    } else if claim, ok := token.Claims.(*TokenClaim); ok {
        return *claim, nil
    } else {
        log.Println(err)
        return TokenClaim{}, err
    }
}
```